

To what extent does the bcrypt algorithm ensure the security of password storage in terms of computational complexity, hashing time, and brute force attacks?

Word Count: 3996

Table of Contents

Introduction	3
Background Information	4
Overview of cryptographic hashing algorithms	4
Investigating the implications of bcrypt	4
The salt parameter	6
Key stretching.....	6
Slow hashing nature of bcrypt	7
Impact of Bcrypt's slow hashing nature	11
Performance and Security Trade-offs	11
Complexity and Processing Time	12
Practical Implications for Security	12
Methodology	12
Experimental Setup & Procedure	12
List of passwords used	12
Data Presentation & Analysis	14
Implications of Findings	16
Bcrypt's cost parameter affects performance.....	17
Impact on High-Volume Systems	17
Balancing Security and Usability	17
Conclusion	18
References	19

Table of Figures

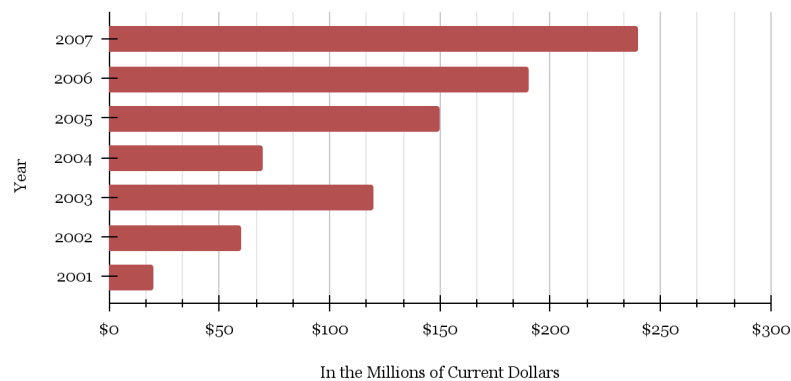
Figure 1, Yearly Amount of Cyber/Internet Fraud	3
Figure 2, Workings of bcrypt	5
Figure 3, Bcrypt Flowchart	6
Figure 4, Impact of varying key length on hashing time	7
Figure 5, Impact of varying iteration counts on hashing time	7
Figure 6, Impact of varying salt value on hashing time	7
Figure 7, Execution time comparison of SHA 512 and bcrypt	8
Figure 8, Time to compute hashes in bcrypt	8
Table I. Comparison of various hashing techniques for reverse hash process using rainbow attacks	9
Table II. Comparison of various hashing techniques for reverse hash process using dictionary attacks	9
Table III a. Alphabet plaintext password hashing results	10
b. Alphabet brute force results	10
Table IV a. Numbers plaintext password hashing results	10
b. Numbers brute force results	10
Table V a. Mixed plaintext password hashing results	10
b. Mixed brute force results	11
Figure 9. Python script to hash password using bcrypt library.....	13
Figure 10. Effects of cost parameter on speed of Bcrypt hashing Password 1	14
Figure 11. Effects of cost parameter on speed of Bcrypt hashing Password 2	14
Figure 12. Effects of cost parameter on speed of Bcrypt hashing Password 3.....	15
Table VI. Cost vs Time	15

Introduction

Password security is the first line of defense for protecting personal information against unauthorized access. Password-based authentication is used to verify the identity of users trying to access a system or service by checking the user's entered username and password credentials against its stored records. If these credentials match, the user is granted access. There are many different types of authentication including single-sign on and hardware keys but passwords remain the most common form. Weak passwords are often vulnerable to malicious software and bad actors and are a major security problem. Thus, encryption and hashing were two methods developed to prevent plaintext passwords from being compromised and authenticate users. In response to increasing cybercrime, the National Institute of Standards and Technology (NIST) provides a guideline for password security in NIST Special Publication 800-63B. Password length should be at least 8 characters, allow passwords with at least 64 characters, avoid common dictionary words or repetitive/sequential characters as well as context-specific words (names, dates of birth, etc). Figure 1 illustrates the amount of money lost each year because of cyber fraud and crime. As cybercrime grows, protecting user passwords via the bcrypt hashing algorithm is critical.

Figure 1

Yearly Amount of Cyber/Internet Fraud



This essay argues that the bcrypt algorithm, due to its adaptive cost factor and resistance to brute force attacks, is one of the most secure options for password hashing, though its performance impact in high-volume systems require careful consideration. Aspects of password security are determined by 1) computational complexity, which is the amount of resources or time an algorithm requires to process and generate a hash, and 2) hashing time, which is the duration it takes for an algorithm to produce a hash value from the input data, and 3) brute force attacks against hashed passwords. This paper also analyzes previous time vs. computation and brute force results from past experiments to further verify which aspects of bcrypt ensure password integrity.

To investigate this relationship, an experiment was conducted to measure bcrypt's performance with varying cost factors and passwords. The hashing time and computational complexity for each password were recorded at different cost parameters.

Background Information

Overview of cryptographic hashing algorithms

A hashing algorithm is a function designed to convert input data (or a data string) into a numeric string of fixed length, known as a hash value or digest. This process enables authentication by verifying only hash values without exposing the original password. This ensures that even if an attacker gains access to the password database, they cannot retrieve the original passwords. Several well known hashing algorithms include MD5, SHA, scrypt, Argon2, and PBKDF2.

The principal security properties¹ of hash functions are:

1. Pre-image resistance: It must be computationally infeasible to retrieve the original input from the hash value.
2. Second preimage resistance: It must be impossible to find an input, which gives a same output as any specified input.
3. Collision resistance: The function guarantees that any alteration to the input results in a different hash and ensures that no two distinct inputs produce the same hash value.

Modern hashes follow an iterative construction proposed by Merkle²/Damgard³. “One-way” hash functions follow properties 1 and 2, whereas “collision resistant” hash functions follow all the properties.

Investigating the Implications of bcrypt

Bcrypt is a cryptographic hash function created in 1999 by Niels Provos⁴ and David Mazières and uses the Blowfish cipher algorithm as its foundation. Every time a user logs in, bcrypt hashes the password anew and compares that hash to the version stored within the password storage to see if they match. Bcrypt is unique because it uses a cost factor, where the number of times the algorithm is run to calculate the final hash (known as password iterations) is set. Higher cost factors use more time and computational power.

Some cryptographic algorithms don't scale with increased computing power, which means faster hardware can quickly break weak hashes. To counter this, bcrypt is designed to run slower on newer hardware, adjusting its complexity to match advances in computing power. This ensures that despite increasing hardware capabilities, the function still can maintain password and security.

Working of the algorithm:

¹ Alfred J. Menezes, Paul C. Van Oorschot, and Scott A. Vanstone. Handbook of Applied Cryptography, chapter Hash Functions and Data Integrity, pages 323–324. The CRC Press series on discrete mathematics and its applications. CRC Press, 1997.

² Ralph C. Merkle. One way hash functions and DES. Lecture Notes in Computer Science, 435:428–446, 1990.

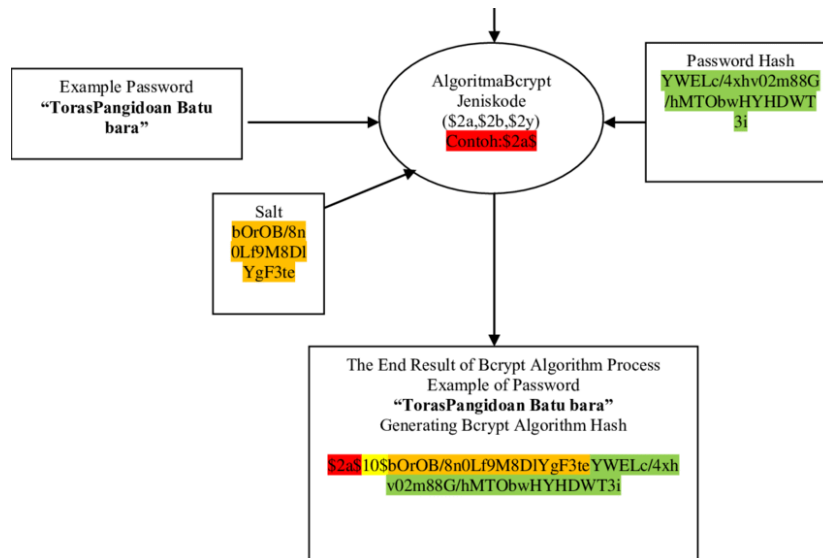
³ I.B. Damgard. A design principle for hash functions. Lecture Notes in Computer Science, 435:416–427, 1990.

⁴ Analysis Performance BCRYPT Algorithm to Improve Password Security from Brute Force (Toras Pangidoan Batubara, Syahril Efendi, Erna Budhiarti Nababan)

1. The raw plaintext password is input.
2. Bcrypt uses the Eksblowfish setup to initialize the encryption process with the provided salt, password, and cost parameter. The setup involves encrypting a constant value ("OrpheanBeholderScryDoubt") 64 times using the Eksblowfish cipher in ECB mode. This phase is time-consuming due to the extensive key schedule.
3. A random salt is generated and combined with the plaintext password. This salt ensures that even if two users have the same password, their hashes will be different.
4. The algorithm processes the salted password through a series of rounds, known as key stretching. The number of rounds is determined by the cost factor (work factor). Increasing the cost factor increases the time required to generate the hash, making brute-force attacks more difficult.
5. The final result of the iterative encryption (hashed password) is concatenated with the salt. This combined hash and salt are stored in the authentication database.

Figure 2 and 3 demonstrate the hashing process visually.

Figure 2: Workings of bcrypt⁵



⁵ Toras Pangidoan Batubara, Analysis Performance BCrypt Algorithm to Improve Password Security from Brute Force

Figure 3: Bcrypt Flowchart⁶

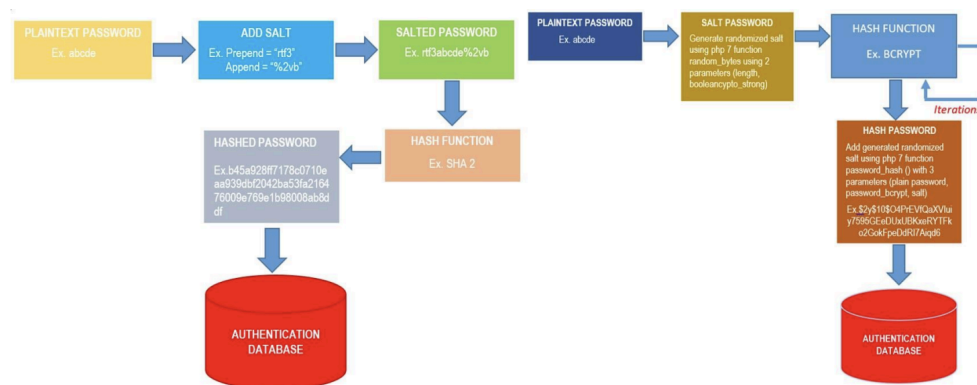


Figure 1a. Existing application of salt and hashing to password

Figure 1b. Proposed application of salt, hashing and iterations to password

The salt parameter

Password salting is a technique used to prevent brute force and rainbow table attacks by adding a randomized string, or “salt,” onto the data before hashing. Each password has its own unique salt. Therefore, even in the case that two users have the same password, the hash digest produced is very different.

Key stretching

Key stretching is a technique that enhances resistance to brute force attacks by requiring more time and computational power to process a password or key. Key stretching strengthens bcrypt's security by deliberately slowing down the password hashing process by using a specialized algorithm that increases computational demands. This intentional delay makes it difficult for high-speed computers to brute force passwords.

⁶ Toras Pangidoan Batubara, Analysis Performance BCrypt Algorithm to Improve Password Security from Brute Force

Slow hashing nature of BCrypt

Study 1, Manpreet Kaur, Implementation and Performance Analysis of PBKDF2, Bcrypt, Script Algorithms

Figure 4, Varying Key Length

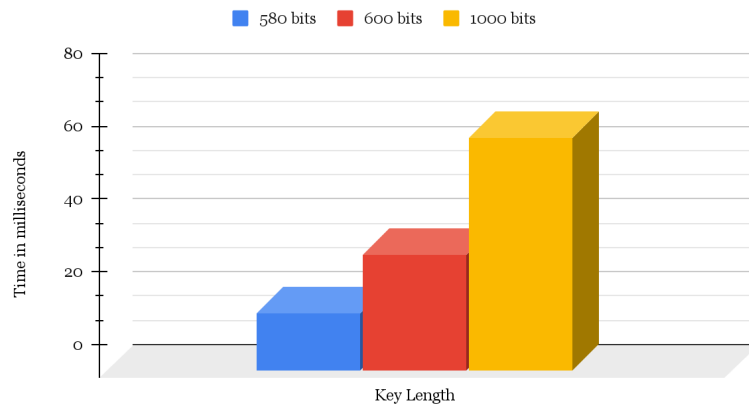


Figure 5, Varying Iteration Counts

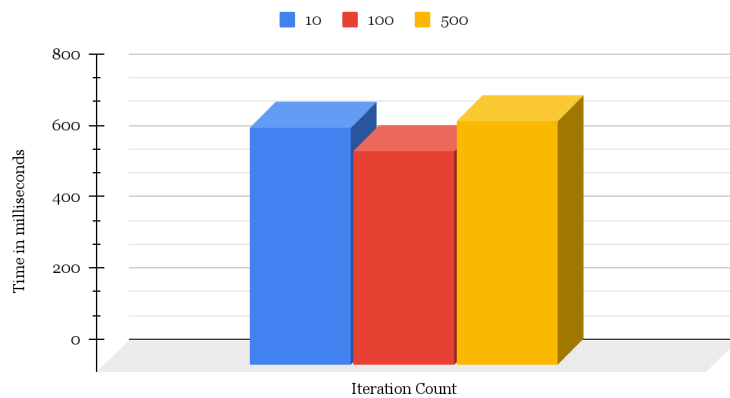
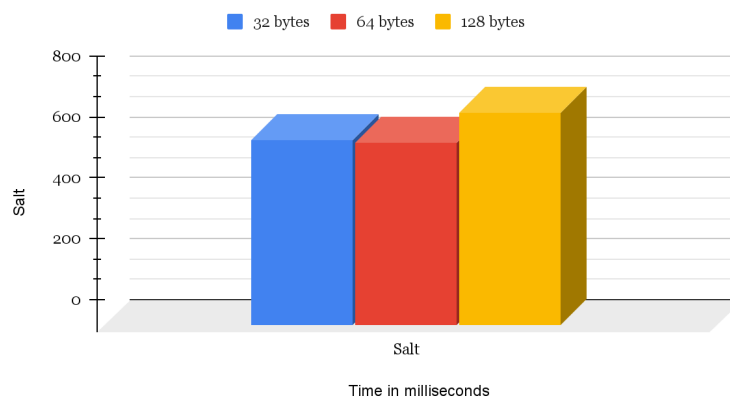


Figure 6, Varying salt value



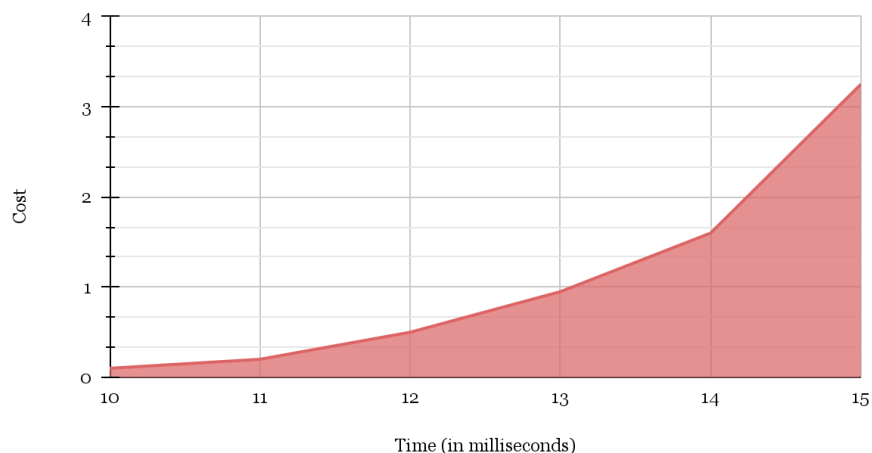
Study 1 demonstrates the performance of Bcrypt with varying key lengths, iteration counts, and salt values. Figure 4 shows a direct correlation between key length and hashing time, with a 1000 bit-long key length resulting in the highest hashing time. Figure 5 shows varying iteration counts influencing hashing time in Bcrypt. A higher cost parameter leads to slower hashing times. This increased computational time enhances security because it makes brute-force attacks, such as those using GPUs, less feasible. Iteration counts of 10 and 500 have higher hashing times than 100. Figure 6 shows that larger salt values, like 128 bytes, also extend hashing times. Additionally, the study found that combining a strong password with BCRYPT provides the most robust password security. Using salted or plain SHA-256 gives similar security, but combining strong passwords with BCRYPT is the most effective method;

Study 2, Roman Alex F. Lustro, Ameliorating Password Security Authentication using bcrypt Algorithm with Dynamic Salt Generation

Figure 8. Execution Time Comparison of SHA 512 and bcrypt

Password	bcrypt (cost = 10)	SHA-512
abc123	0.07621097565	0.0009500980377
letmein	0.07513189316	0.0008578300476
trustno1	0.074187994	0.001965004593
p@\$w0rd1234	0.07768702507	0.0010251999
J3llyFish	0.07677412033	0.0007708072662
AllBlack\$1	0.07675004005	0.001084089279

Figure 9, Time to compute hashes in bcrypt



While SHA-512 is faster, it is more vulnerable to attacks because attackers can attempt more hashes per second. Bcrypt, however, takes longer for a threat actor to act because it uses a flexible number of password iterations, which evolves with hardware and software optimization.

Study 3, Rainbow Attack and Dictionary Attack on Bcrypt-Hashed Passwords⁷

Table I. Comparison of various hashing techniques for reverse hash process using rainbow attacks

Hashing Technique	Total Time (ms)	Avg. Time (ms)	State
Bcrypt	N/A	N/A	Not Successful
MD-5	0.11	0.022	Successful
SHA-256	0.18	0.036	Successful

Table II. Comparison of various hashing techniques for reverse hash process using dictionary attacks

Hashing Technique	Total Time (ms)	Avg. Time (ms)	State
Bcrypt	N/A	N/A	Not successful
MD-5	0.13	0.026	Successful
SHA-256	0.27	0.054	Successful

Study 4, Toras Pangidoan Batubara, Analysis Performance BCRYPT Algorithm to Improve Password Security from Brute Force

Table III. a. Alphabet plaintext password hashing results

Algorithm Code	Character Type	Number of Characters	Cost	Plaintext Password	Password Results
\$2b\$	Alphabet	5	10	toras	\$2b\$10\$9eiAF6l3no JdpYrz9PQl Ku2sUOU Cv/XoscmA9Pizc3 Hci7UXrm4x C

Table III. b. Alphabet brute force results

Algorithm Code	Character Type	Number of Characters	Password	Password Bcrypt	Time	Brute Force Results
----------------	----------------	----------------------	----------	-----------------	------	---------------------

⁷ (Skanda C, Secure Hashing using Bcrypt for Cryptographic Applications)

\$2b\$	Alphabet	5	-	\$2b\$10\$9eiAF6l 3noJdpYrz9PQl Ku2sUOU Cv/XoscmA9Piz c3Hci7UXrm4x C	5 days	Not found
--------	----------	---	---	---	--------	-----------

Table IV. a. Numbers plaintext password hashing results

Algorithm Code	Character Type	Number of Characters	Cost	Plaintext Password	Password Results
\$2b\$	Numbers	7	10	1234567	\$2b\$10\$1M77axV QRHE7M9odbfleI .esW3g/K4D6lElH 9Gewh0pHqDBAy q1JG

Table IV. b. Numbers brute force results

Algorithm Code	Character Type	Number of Characters	Password	Password Bcrypt	Time	Brute Force Results
\$2b\$	Numbers	7	1234567	\$2b\$10\$1M77a xVQRHE7M9o dbfleI.esW3g/ K4D6lElH9Ge wh0pHqDBAy q1JG	8 hours	Found

Table V. a. Mixed plaintext password hashing results

Algorithm Code	Character Type	Number of Characters	Cost	Plaintext Password	Password Results	
\$2b\$	Mixed	7	12	toras3\$	\$2b\$12\$B3GYY 8jB/.kj9ealyuhay e20inr VCDaC1vPkkcH 1s95bgnyV3olu O	

Table V. b. Mixed brute force results

Algorithm Code	Character Type	Number of Characters	Password	Password Bcrypt	Time	Brute Force Results
----------------	----------------	----------------------	----------	-----------------	------	---------------------

\$2b\$	Mixed	7 -	\$2b\$12\$B3GYY 8jB/.kj9ealyuhay e20inr VCDaC1vPkKcH 1s95bgnyV3olu O	5 days	Not found
--------	-------	-----	---	--------	-----------

While increased hashing times improve security, they also impact system performance, especially on online platforms where there is a large amount of login attempts. So there needs to be a balance between cost parameters and the time taken to generate such hash. Additionally, the research shows that the bcrypt Algorithm performs well against Brute Force attacks on alphabetic and mixed characters. However, it's not as effective in preventing brute force attacks on numeric characters;

As demonstrated within Study 2, even a simple password such as “toras” can not be brute forced through bcrypt due to its slow computational hashing and dynamic salting. When taken with Study 1, where bcrypt was unable to be reverse hashed through Rainbow tables, it becomes clear that bcrypt is less vulnerable to rainbow and dictionary attacks due to its slow hashing computation and cost parameter (as evidenced with the salt parameter and key stretching as well).

Impact of Bcrypt’s Slow Hashing Nature

Study 1 demonstrates that bcrypt’s hashing time significantly increases with key length and iteration count. For instance, with an iteration count of 10, the time to compute hashes in bcrypt can extend to approximately 0.0767 milliseconds (as seen in the chart data), compared to SHA-512’s much quicker 0.00096 milliseconds. A higher cost parameter, such as 12 or 14, exponentially increases this time, making brute-force attacks less feasible. For example, increasing the cost parameter from 10 to 12 can lead to a hashing time increase from around 0.0767 milliseconds to 0.1487 milliseconds which effectively doubled the time taken to hash passwords.

Study 3 confirms that bcrypt is highly resistant to rainbow and dictionary attacks due to its slower hashing and effective salting. Unlike MD-5, which has an average hashing time of 0.11 milliseconds and is vulnerable to such attacks, Bcrypt’s computational cost and dynamic salting make it significantly more secure. For example, MD-5 was unsuccessful in preventing attacks during tests, while bcrypt consistently secured passwords.

Performance and Security Trade-offs

Study 4 demonstrates that while Bcrypt’s increased computational time enhances security against brute-force attacks, it also impacts system performance. With a cost parameter of 12, Bcrypt needs 0.1487 milliseconds to hash a password, compared to SHA-256’s consistent 0.056 milliseconds. Higher cost parameters, while improving security, can result in a performance trade-off, especially in high-demand environments where rapid login attempts are frequent. The chart data supports this by showing that as the cost increases, the time required to process passwords increases exponentially.

Complexity and Processing Time

Chart data also reveals that password complexity (inclusion of numbers or special characters) does not significantly affect processing time at a given cost level. Study 1 supports this observation, showing that while password complexity can indeed enhance security, Bcrypt's hashing time is primarily influenced by cost parameters. For example, a complex password like "p@55w0rd1234" may take 0.0787 milliseconds to hash at a cost of 10, similar to a simpler password.

Practical Implications for Security

Studies 2 and 4 highlight that Bcrypt is particularly effective in making brute-force attacks impractical due to its slow hashing process and dynamic salting. For instance, Study 4 notes that while Bcrypt is effective in securing alphabetic and mixed character passwords (taking up to 5 days to attempt a brute-force attack), it is less effective for numeric passwords. Thus, combining password complexity with Bcrypt's slow hashing is the most optimal way to maximize password security.

Methodology

Experimental Setup & Procedure

This study utilizes Python as the primary programming language and the built-in 'bcrypt' library for password hashing and the 'time' module to measure hashing duration. The experiments are conducted on a MacBook Pro (16-inch, November 2023) equipped with an Apple M3 chip, 18 GB of memory, and a 12-core CPU, running macOS Sonoma 14.2.1. The Python scripts are executed within the Visual Studio Code environment. Each password was hashed five times for every cost factor, and the average hashing time was recorded to reduce variability and ensure consistent results.

A Python script is designed to hash passwords using 'bcrypt.' As a control, we use three passwords with varying complexities and combinations of numbers, letters, and special characters. The independent variables are every possible cost factor (4-32) used for salt generation. The dependent variable is the time taken to hash the password, CPU usage, and memory usage. The hardware and software environment are kept constant.

List of passwords used:

- Password 1 (cocacolakoala) represents a weak and insecure password containing only letters.
- Password 2 (cocacolakoala123456789) represents a relatively secure password containing letters and numbers.
- Password 3 (c0c@c0l@ko@l@12345!!) represents a very secure password containing letters, numbers, and special characters.

Figure 9, Python script to hash password using Bcrypt library

```
import bcrypt
import time

def hash_password(password, cost):
    start_time = time.time()
    salt = bcrypt.gensalt(rounds=cost)
    hashed = bcrypt.hashpw(password.encode(), salt)
    end_time = time.time()
    duration = end_time - start_time
    return hashed, duration

passwords = ["cocacolakoala", "cocacolakoala123456789", "c0cc0l@ko@la@12345!!"]
cost_factors = list(4,32)
results = []

for password in passwords:
    for cost in cost_factors:
        hashed_pw, duration = hash_password(password, cost)
        results.append(password, cost, hashed_pw, duration)

for result in results:
    print(f"Password: {result[0]}, Cost: {result[1]}, Hashing Time: {result[3]:.4f} seconds")
```

Data Presentation & Analysis

Results

Figure 10. Effects of cost parameter on speed of Bcrypt hashing Password 1

Effect of cost parameter on speed of Bcrypt hashing Password 1

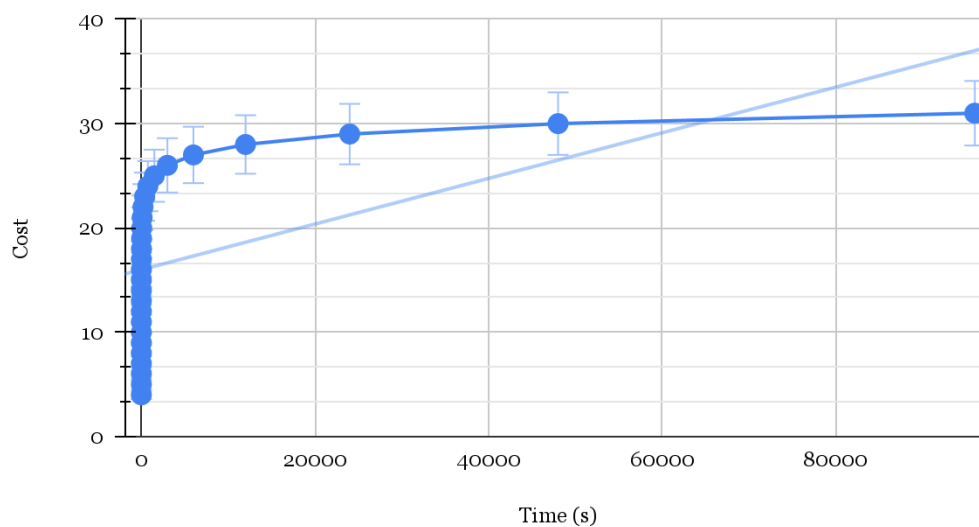


Figure 11. Effects of cost parameter on speed of Bcrypt hashing Password 2

Effect of cost parameter on speed of Bcrypt hashing Password 2

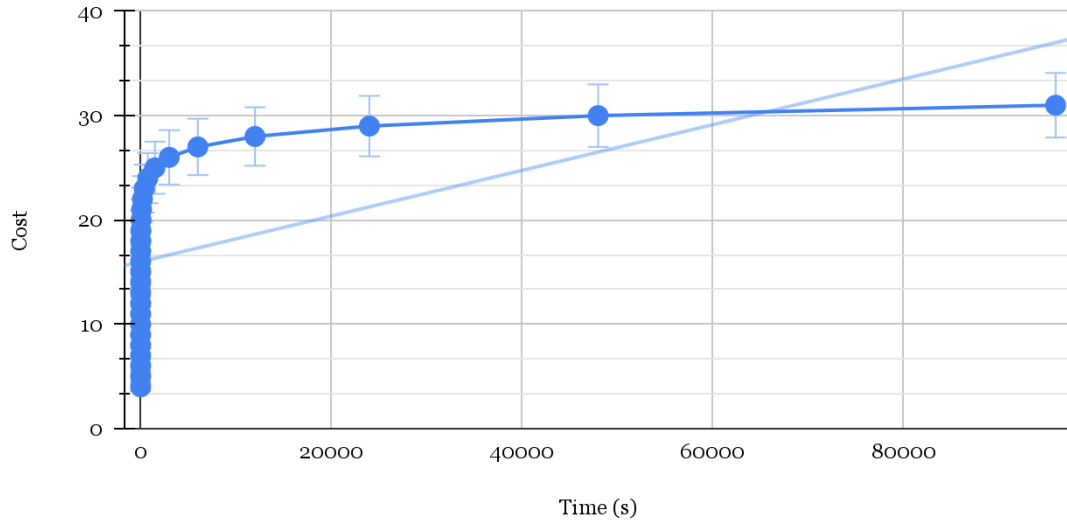


Figure 12. Effects of cost parameter on speed of Bcrypt hashing Password 3

Effect of cost parameter on speed of Bcrypt hashing Password 3

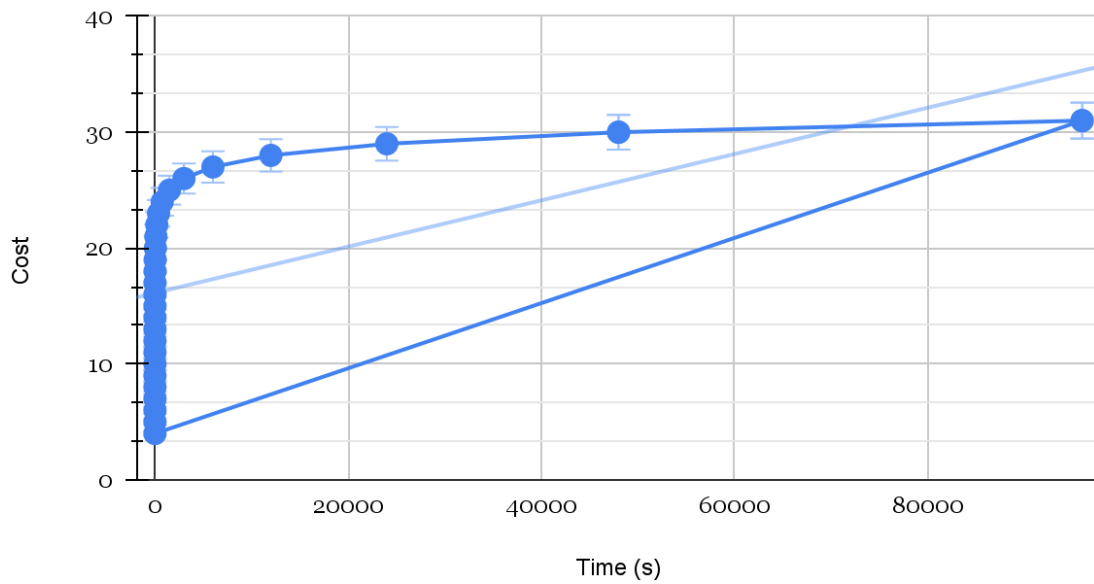


Figure 13. Password Hashing Times

Cost Parameter	Password A (s)	Password B (s)	Password C (s)
4	0.00077	0.00082	0.00081
5	0.00183	0.00185	0.0018
6	0.00292	0.0029	0.00304
7	0.006	0.00595	0.00585
8	0.01167	0.0117	0.01173
9	0.02365	0.0238	0.0239
10	0.04674	0.04687	0.0467
11	0.09385	0.09375	0.0931
12	0.18836	0.18694	0.18657
13	0.37519	0.37381	0.3732
14	0.73465	0.74462	0.7355
15	1.43888	1.46312	1.46275
16	2.93808	2.95417	2.93786
17	5.85872	5.91212	5.86374
18	11.72318	11.74262	11.71606
19	23.38212	23.425	23.43512
20	46.893	46.85	46.87024
21	93.572	93.7	93.74048
22	187.573	187.4375	187.48096
23	375.291	374.875	374.96192
24	750.292	749.75	749.92384
25	1500.585	1499.5	1499.84768
26	3000.468	2999	2999.69536
27	6002.357	5998	5999.39072
28	12005.316	11996	11998.78144
29	24010.632	23992	23997.56288
30	48021.264	47984	47995.12576
31	96042.528	95968	95990.25152

Implications of Findings

For all three passwords, the time required to process the password increases exponentially as the cost increases. Computation cost has a significant impact on the time required to process passwords, which is expected as higher computational cost usually involves more iterations of hashing algorithms. The time differences between the three passwords at each cost level are negligible as the length and inclusion of special characters (indicating that the complexity of the password) does not significantly impact the time required for processing at a given computational cost. The inclusion of the longer password (cocacolakoala123456789) compared to the shorter password (cocacolakoala) does not significantly change the processing time. This suggests that the length of the password within these examples is not a critical factor in the hashing process for the measured cost levels. This aligns with the findings of Study 4.

While password complexity doesn't affect hashing time significantly, the cost parameter does on a logistic scale. Shown in all three passwords, hashing time increased very rapidly at first, but started leveling out later on. This pattern likely continues to increase with higher cost parameters. Although the password complexity does not affect the processing time in this chart, in real-world applications, more complex passwords with a mix of characters, numbers, and symbols (like c0c@c0l@ko@l@12345!!) are much more resistant to brute-force attacks, even though they take the same time to process.

Cost factor of 4 has hashing ranges of 0.00077 to 0.00082 seconds across all three passwords. Cost factor of 18 has a hashing average of approximately 11.72 seconds across all three passwords. The progression of time from cost factors 4 through 18 shows a steep increase, particularly from cost factors 12 onwards, where the time taken increases sharply.

Bcrypt's cost parameter affects performance

The cost parameter in bcrypt is an exponential factor that controls the number of key expansion iterations. For example, if the cost is set to 4, bcrypt performs $2^4 = 16$ iterations; if set to 18, it performs $2^{18} = 262,144$ iterations. The primary impact of a higher cost parameter is a significant increase in hashing time. As shown in this experiment, processing the password "cocacolakoala" took only 0.00077 seconds at cost 4, but this time escalated dramatically to 11.72318 seconds at cost 18. This exponential increase demonstrates how higher cost factors contribute to greater computational complexity, making brute-force attacks much less feasible because attackers would need significantly more time and resources to crack a single password.

Impact on High-Volume Systems

In systems where user logins occur frequently, higher hashing times can lead to delays and reduced system responsiveness. For instance, if a website has a high traffic volume with millions of users and passwords to verify, each login attempt will require more processing power and time as the cost parameter increases. This results in slower response times, increased server load, and potentially higher costs for maintaining the infrastructure. The main goal of bcrypt is to introduce quick and secure password hashes.

Balancing Security and Usability

While it is tempting to set a very high cost parameter to maximize security, doing so will compromise system efficiency and user experience. It is therefore essential to find the right balance between security and performance. Extremely high cost parameters can slow down password verifications and result in longer login times and increased server load. For companies managing high traffic or large-scale systems, this could lead to delays, higher energy consumption, and even user frustration. On the other hand, setting a low cost parameter improves user efficiency but weakens security and user data by making systems more vulnerable to brute-force attacks.

Conclusion

In this paper, the bcrypt algorithm was analyzed in terms of computational complexity, hashing time, and brute force attacks through evaluation of previous experiments and the current experiment. Thus, the research question, “To what extent does the bcrypt algorithm ensure the security of password storage in terms of computational complexity, hashing time, and brute force attacks?” has been answered. As depicted in Figure 1, the increasing frequency of breaches and cybercrime attacks shows the importance of utilizing hashing algorithms to protect user information. Bcrypt is well known to adapt with contemporary threats and technology through features such as dynamic salting and key stretching, which enhances its security against evolving technology and attack methods. Bcrypt is particularly resistant to brute force attacks, one of the security criteria, due to its slow hashing nature, coupled with its use of dynamic salting, enhances security by making brute-force and dictionary attacks less feasible. Study 1 and Study 2 confirm that Bcrypt’s increased hashing times improve security, as attackers face higher computational costs. Bcrypt’s resistance to rainbow and dictionary attacks, as demonstrated in Study 3, further validates its effectiveness. Additionally, the analysis shows that password complexity (e.g., including special characters) does not significantly affect processing time at a given cost level. However, using complex passwords with Bcrypt significantly enhances security. Our results support this, as data clearly shows that as the computational cost parameter increases, the time required to process passwords also increases exponentially.

While bcrypt ensures passwords remain secure, there is one trade-off in performance. The data from Study 4 highlights the trade-off between security and performance. Higher cost parameters lead to slower processing times, which, although improving security, can affect systems with high login volumes. Thus, in large scale websites or systems with millions of users, high cost factors can require high volumes of password verifications, energy consumption, and time. It is therefore important to find a balance in cost factor without compromising user login time and password security.

On a personal level, this research taught me a great deal about the delicate balance between security and performance in password management. I initially chose this topic out of curiosity about how well-known hashing algorithms like bcrypt handle the growing sophistication of attacks. I realized how crucial it is for both companies and users to be aware of the implications of password security. Companies, in particular, have to make difficult choices when scaling up their systems, as using high-cost parameters can increase operational demands. However, I discovered bcrypt is best suited to protect user data while striking the perfect balance between user efficiency and security. One potential area for further study is by comparing bcrypt against a more modern algorithm, such as Argon2, which has emerged as a stronger competitor in recent

years. Argon2 offers more flexibility in memory usage and processing times, which might be valuable for future password security applications.

References

- [1]S. C, S. B, and P. B.S., “Secure Hashing using BCrypt for Cryptographic Applications,” Nov. 20, 2022. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10126956> (accessed Apr. 24, 2024).
- [1]R. A. Lusto, “Ameliorating Password Security Authentication Using BCRYPT Algorithm with Dynamic Salt Generation,” Journal of Advanced Research in Dynamical and Control Systems, Dec. 2019. https://www.researchgate.net/profile/Roman-Alex-Lusto/publication/338480957_Ameliorating_Password_Security_Authentication_Using_BCRYPT_Algorithm_with_Dynamic_Salt_Generation/links/633f809a2752e45ef69a0385/Ameliorating-Password-Security-Authentication-Using-BCRYPT-Algorithm-with-Dynamic-Salt-Generation.pdf (accessed Apr 24, 2024).
- [1]L. Ertaul, M. Kaur, and V. A. K. R. Gudise, “Implementation and performance analysis of pbkdf2, bcrypt, scrypt algorithms,” 2016. <http://mcs.csueastbay.edu/~lertaul/PBKDFBCRYPTCAMREADYICWN16.pdf> (accessed Apr. 24, 2024).
- [1]S. P, “PROVIDING PASSWORD SECURITY BY SALTED PASSWORD HASHING USING BCRYPT ALGORITHM ,” ARPN Journal of Engineering and Applied Science, Jul. 2015. <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=9d82620f0866ec773b12c30f90d70b74661f972f> (accessed Apr. 24, 2024).
- [1]T. P. Batubara, S. Efend, and E. B. Nababan, “Analysis Performance BCRYPT Algorithm to Improve Password Security from Brute Force,” Journal of Physics. <https://iopscience.iop.org/article/10.1088/1742-6596/1811/1/012129/pdf> (accessed Apr. 24, 2024).